

14. Modelos de clasificación

Ciencia de datos para la toma de decisiones II

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Matriz documento-término

Una **matriz documento -término (Document -Term Matrix, DTM)** es una matriz en la que cada columna representa un término, palabra o token, cada fila representa un documento (artículo, tweet o párrafo) y cada celda contiene el número de veces que ese término aparece en ese documento (frecuencia) o una ponderación (como TF-IDF).

Doc\Término	economía	inflación	crecimiento
Doc1	3	1	0
Doc2	2	0	4
Doc3	1	2	1

Función `tidytext::cast_dtm()`

Es la función que nos ayuda a pasar de una tabla tidy de palabras-documento (la que generamos con el `unnest_token()`) a una matriz término-documento.

```
matriz_dtm <- texto_token %>%  
  cast_dtm(document = id,  
           term = palabras,  
           value = n,  
           weighting = tm::weightTfIdf)
```



Como se ve

```
> matriz_dtm %>% as.matrix() %>% as_tibble()  
# A tibble: 690 × 11  
  gente violencia videojuegos juegos impuestos hacer  
  <dbl>      <dbl>      <dbl>  <dbl>      <dbl> <dbl>  
1 0          0          0      0          0      0  
2 0.554      0          0      0          0      0  
3 0          0.541      0      0          0      0  
4 0          0.0971     0.106 0.0493      0      0  
5 0          0          0      0          3.13 0  
6 0          0          0      0          0      0.269  
7 0          0          0      0          0      0  
8 0.117      0          0.167 0          0      0.122  
9 0          0          0      0          0      0  
10 0          0          0      0          0      0
```

```
<<DocumentTermMatrix (documents: 690, terms: 11)>>  
Non-/sparse entries: 759/6831  
Sparsity             : 90%  
Maximal term length: 11  
Weighting            : term frequency - inverse document frequency (normalized) (tf-idf)
```



Convertida en *matriz* y luego en *tibble*

Función *tidytext::cast_dtm()*

Argumentos:

- o **document_id**: La columna que identifica al documento.
- o **term**: la columna que contiene las palabras
- o **value**: la columna que contiene los valores de las celdas de la matriz. Pueden ser frecuencias o ponderaciones *tf-idf*.

weighting: Ponderación de la matriz

```
matriz_dtm <- texto_token %>%  
  cast_dtm(document = id,  
           term = palabras,  
           value = n,  
           weighting = tm::weightTfIdf)
```

Matríces dispersas

Es una matriz en la que la **mayoría de sus elementos son cero.**

- Si una matriz es 99% dispersa, significa que el 99% de sus elementos es cero.
- Esta característica de **dispersión** es una característica muy común de las matrices documento término, ya que hay tantas palabras en los corpus que es muy común que haya muchos ceros en los vectores que representan a los documentos.

[illegible]

Matrices dispersas p.2

En procesamiento de texto, las matrices dispersas **son la norma:**

- Un **vocabulario** puede tener **decenas de miles de palabras**.
- Pero cada documento solo usa unas **cuantas centenas**.
- Si guardas todos los ceros, desperdicias **mucha memoria y tiempo de cómputo**.

Matrices dispersas p.2

La métrica de la dispersión, que (en este caso) es una métrica asociada con las matrices de documento término, **se puede reducir** con diversas funciones, como la función ***tm::removeSparseTerms()***.

Con esta función, podemos reducir la dispersión en un tanto por ciento, descartando (de forma arbitraria) información dentro de nuestra matriz de análisis.

Decidir sobre la dispersión de la matriz depende de cuántos términos están en tu matriz y qué tan rápida es tu computadora.

★ ***Etiquetado con LLM***

Es el proceso de **asignar etiquetas automáticas** a textos mediante un modelo de lenguaje de gran escala (LLM).

En lugar de reglas o clasificadores tradicionales, se aprovecha la *comprensión semántica* del modelo para generar juicios contextuales (por ejemplo: “Ofensivo / No ofensivo”).

Ejemplo:

Prompt: “¿Consideras que este comentario es ofensivo?”

Texto: “Ese juego es una pérdida de tiempo.”

Respuesta del modelo: *No ofensivo.*

★Etiquetado con LLM p.2

Ventajas:

- Rápido y flexible ante nuevos criterios.
- Requiere poco o ningún entrenamiento previo.
- Puede adaptarse fácilmente cambiando el *prompt*.
- Puede entenderse el proceso por el cual se llegó a la clasificación

Limitaciones:

- Variabilidad en las respuestas (entre intentos y entre modelos).
- Necesidad de normalizar o limpiar salidas (manualmente o con expresiones regulares)
- Dependencia del modelo y su sesgo contextual.

★ *Flujo técnico del etiquetado con LLM*

(Ollama + R)

Comentarios (Excel) → R (httr2/jsonlite) → Ollama (localhost:11434/api/generate)



JSON con respuesta ("Ofensivo"/"No ofensivo")



Limpieza y exportación a Excel

```
prompt_text <- str_c("¿Consideras que este comentario es ofensivo? Estaba en un  
video en el que se discutía la nueva propuesta de impuestos a videojuegos violentos  
para el próximo año:", "", comentario, "'. Solo menciona si es 'Ofensivo' o 'No  
ofensivo', sin dar explicaciones")  
  
# Construimos la petición HTTP a Ollama  
resp <- request("http://localhost:11434/api/generate") |>  
  req_body_json(list(  
    model = "llama3.1:latest", # Nombre exacto del modelo que tienes  
    prompt = prompt_text,  
    stream = FALSE # FALSE para obtener todo el texto de una vez  
  )) |>  
  req_perform()  
  
# Procesamos la respuesta JSON  
result <- resp_body_json(resp)  
result$response
```

★ *Modelos de clasificación*

El modelado de clasificación es el proceso de aprendizaje supervisado de entrenar algoritmos para **predecir categorías o clases** de nuevos documentos **basándose en patrones aprendidos** de documentos **etiquetados**.

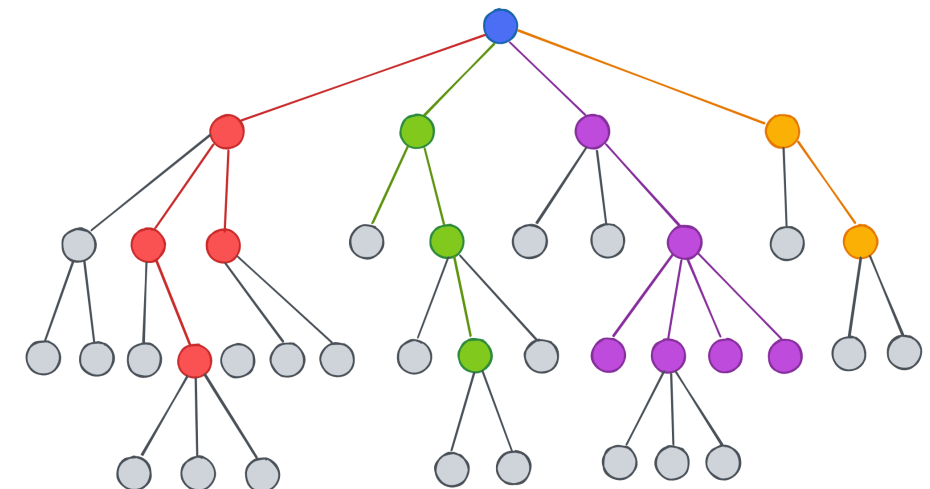
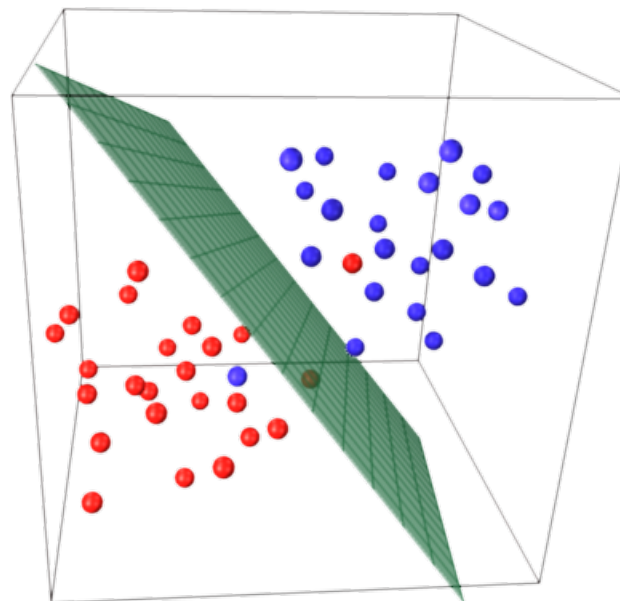
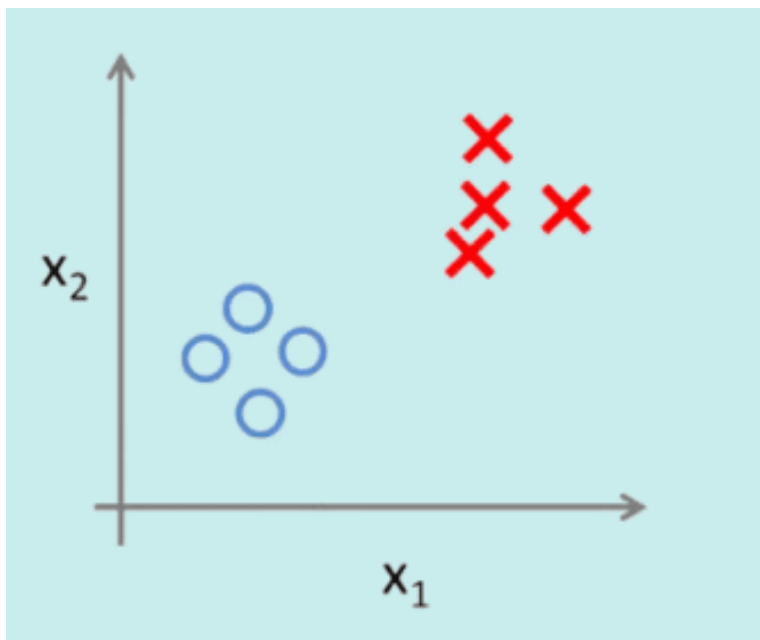
Es una de las aplicaciones más prácticas del análisis de texto en el mundo real.

Características del modelado de clasificación:

- Enfoque de aprendizaje supervisado
- Distingue entre diferentes categorías
- Requiere datos etiquetados para entrenamiento
- Puede predecir categorías de nuevos documentos

Algunos modelos de clasificación:

- Regresión logística
- Árboles de decisión, bosques aleatorios,
- Xgboost
- Redes Neuronales
- Support Vector Machines
- Entre otros.



Pasos básicos para usar texto en modelos de Machine Learning

1. Limpia/prepara los datos
2. Crea una base de entrenamiento y una base de prueba/validación
3. Entrena el modelo en la base de entrenamiento
4. Analiza la exactitud del modelo en la base de prueba/validación

randomForest::randomForest()

Función que implementa el algoritmo del modelo de bosques aleatorios para regresión y clasificación.

Argumentos:

x: Matriz de variables explicativas (en el caso de análisis de texto, estas serían las ponderaciones de la matriz documento-término, además de características adicionales).

y: Vector de etiquetas (categorías en los que hay que clasificar a los documentos).

ntree: (Hiperparámetro) Número de arboles a generar.
Recomendación: 100 arboles es bueno para una primera iteración rápida, 200-300 es lo normal y 500-1000 cuando se quiere expresar toda la exactitud posible.

Ejercicio 01

1. Genere un modelo de clasificación de comentarios sobre el video de ***El Gobierno de México quiere cobrar impuesto a videojuegos violentos: ¿Es justificable?*** (<https://www.youtube.com/watch?v=6KQNqfr5F3A>). Para ello:
 1. Cargue los datos del archivo de excel de la carpeta 01_Datos. Estos datos ya vienen etiquetados como “Ofensivo” y “No ofensivo”, dependiendo de cada mensaje.
 2. Obtenga los n-gramas de los términos más comunes e inclúyalos como palabras clave
 3. Limpie los términos que considere que no aportan información a los datos.
 4. Tokenice por palabras y elimine las palabras comunes (stop_words).
 5. Obtenga la matriz documento-término con los comentarios

Ejercicio 01

Ejercicio

6. A partir de la matriz de documento-término, obtenga la matriz de variables X y el vector de etiquetas Y necesarias para el modelo de clasificación.
7. Realice el modelo de clasificación. Utilice el modelo ***randomforest::randomForest()***
8. Evalúe, a partir de la matriz de confusión, la exactitud del modelo.

Mejora de resultados del modelo

Algunas cosas que se podría hacer para mejorar la exactitud del modelo:

1. Modificar inteligentemente los stop
2. Generar un diccionario de palabras ofensivas
3. Ajustar la dispersión de la matriz dtm
4. Agregar **características adicionales** (más variables de X) a la matriz dtm. Como características de sentimientos.
5. Cambiar el modelo de clasificación o armar un ensamble de modelos.
6. Modificar los hiperparámetros.